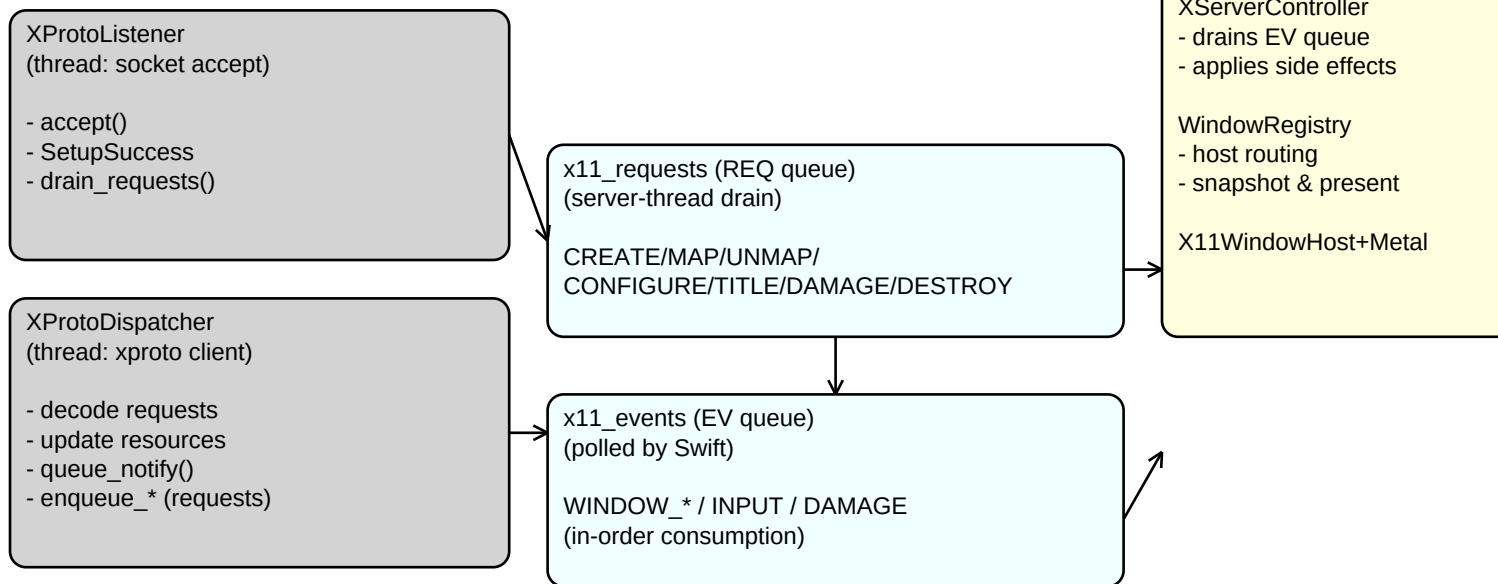


SwiftX11 / X11LowLevel — Refactor diagrams (Step 7-1..7-4)

These diagrams model the CURRENT design, plus a clean C++ refactor target to keep XProtoDispatcher maintainable.

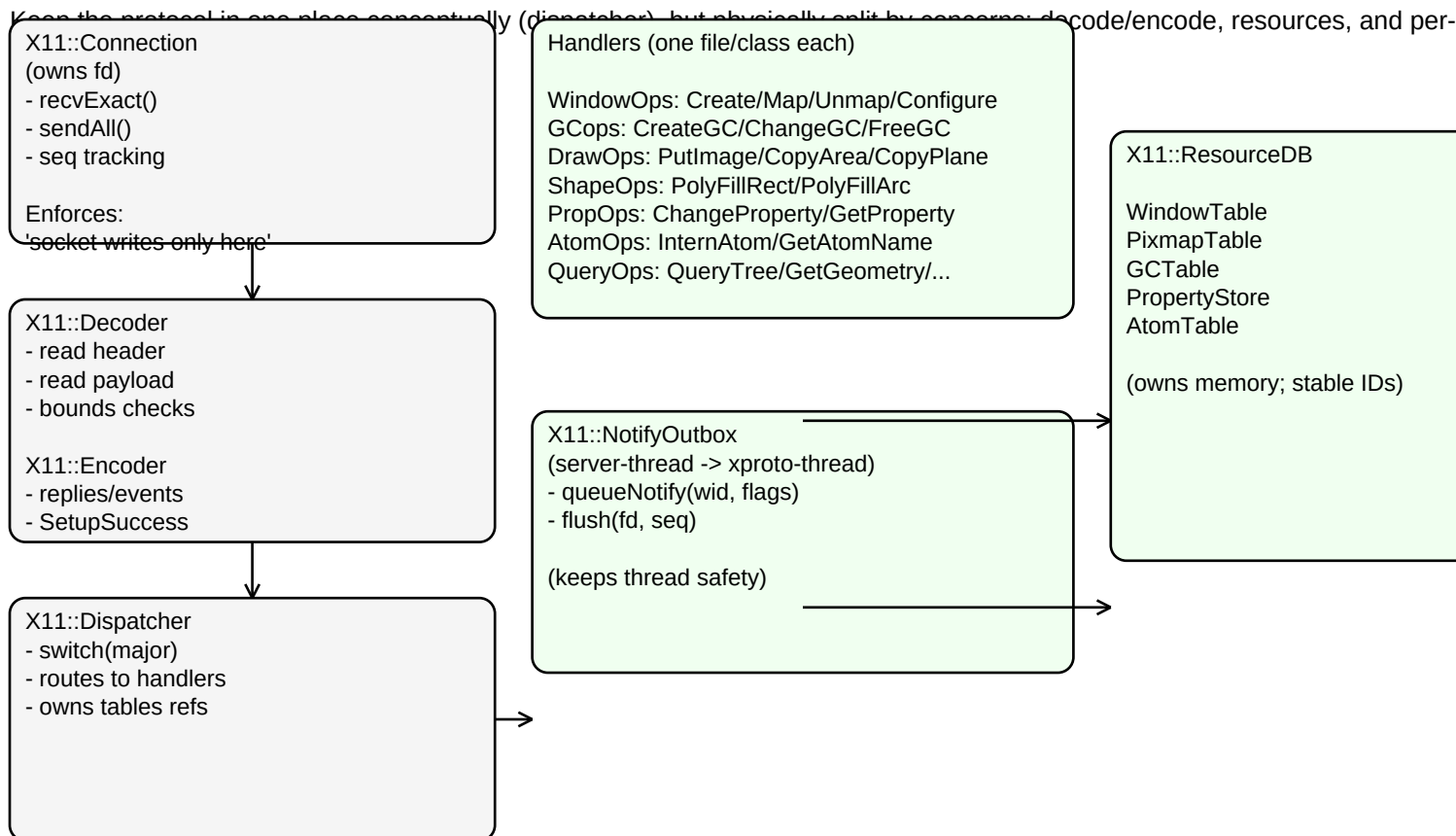
7-1. High-level architecture (threads + dataflow)



Key rule: ONLY the xproto client thread writes to the client socket. Server-thread work queues notifications via queue_notify().

Goal of refactor: split 'x11_xproto.c' into cohesive C++ components while preserving this thread rule.

7-2. Proposed C++ decomposition of XProtoDispatcher



Maintainability tactic: each opcode family becomes a small handler with a single unit-testable function.
Dispatcher stays as a thin router; no resource/layout logic inside the switch.

Window / Pixmap / GC

Window (XID)
- parentXid
- x,y,w,h
- mapped/presentable/dirty
- eventMask
- owner_fd

BackingStore:
- fb (ARGB32)
- resize preserves overlap

Pixmap (XID)
- depth
- w,h

If depth==1:
- bits (packed)
- strideBytes (32-bit pad)

Else:
- pixels (ARGB32)

(a) Window and Pixmap are stored packed; (b) CopyPlane uses GC fg/bg mapping into 32bpp window.

GC (XID)
- fg (ARGB)
- bg (ARGB)

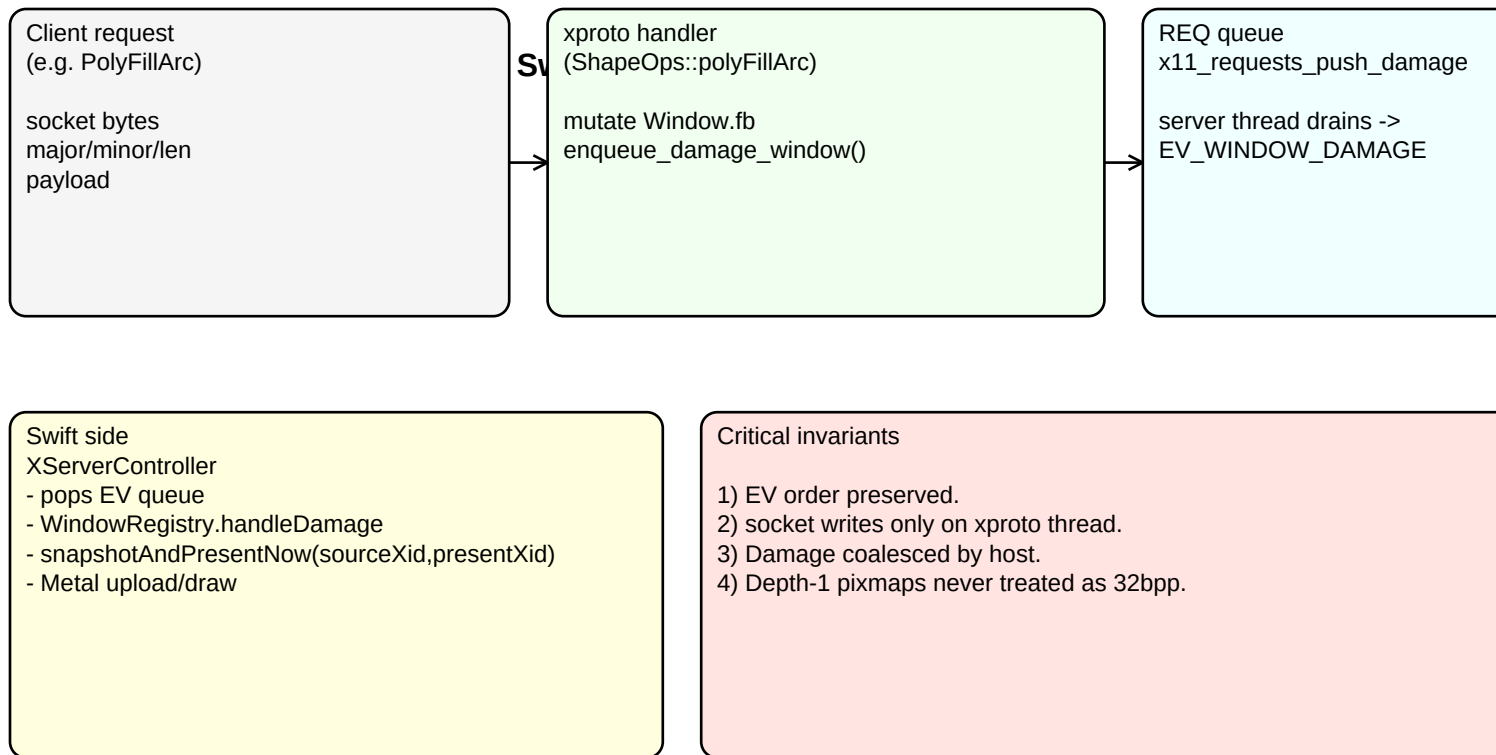
Used by:
- PutImage(XYPixmap)
- CopyPlane
- PolyFill*

Draw pipeline

PutImage -> Pixmap(bits)
CopyPlane -> Window(pixels)
PolyFill* -> Window(pixels)
Damage -> Swift snapshot

Refactor goal: move these structs into C++ classes but keep data layout explicit (no hidden allocations in hot paths).

7-4. Event/request flow + keeping XProtoDispatcher maintainable



Keeping XProtoDispatcher maintainable:

- Split by opcode family (WindowOps / DrawOps / ShapeOps / PropOps / AtomOps / QueryOps).
- Each handler takes (Connection&, ResourceDB&, Outbox&, RequestView) and returns void.
- Put all structs for a request/reply in one header; keep packing/endianness helpers centralized.